

E-commerce Purchase Prediction: Big Data End-to-End Pipeline

Team 9: Albina, Venera

May 2026

Contents

1	Business Understanding	3
1.1	Problem Context	3
1.2	From Business Objective to Modelling Metric	3
1.3	Unit Economics of the Selected Model	3
2	Data Understanding and Exploratory Analysis	4
2.1	Dataset Description	4
2.2	Class Imbalance	4
2.3	Session Funnel	4
2.4	Temporal Patterns	5
2.5	Brand Performance and Conversion	6
3	Data Preparation	6
3.1	Ingestion Pipeline	6
3.2	Storage Format Selection	6
3.3	Partitioning and Bucketing	6
3.4	Technical Fixes	7
4	Modelling and Memory Management	7
4.1	Comparative Balancing Experiment	7
4.2	Hyperparameter Tuning	8
4.3	Memory and Resource Management	8
4.4	Software Engineering Standards	9
5	Evaluation and Risk Assessment	9
5.1	Model Comparison	9
5.2	Model Selection Rationale	9
5.3	Feature Importance	10
5.4	Risk Assessment: Price-Surge Simulation	10
5.5	ML Performance Visualisation	11

6	Deployment and Deliverables	12
7	Team Participation	13

1 Business Understanding

1.1 Problem Context

Online multi-category retail platforms suffer from a persistent and economically significant phenomenon known as *cart abandonment*. In the dataset under study, only 1.75% of recorded user events correspond to a completed purchase; the remaining 98.25% are non-revenue-generating browsing actions (**view**, **cart**). The operational objective is therefore to identify, at the level of an individual session and ideally in near real-time, those users whose behavioural signal indicates a high probability of completing a purchase, so that retention interventions (targeted promotions, abandonment emails, retargeting impressions) can be allocated selectively rather than uniformly.

1.2 From Business Objective to Modelling Metric

A degenerate classifier that predicts **not-purchase** for every event achieves 98.25% Accuracy on this dataset while identifying zero potential buyers. Accuracy is therefore demonstrably uninformative as a model-selection criterion under heavy class imbalance and was explicitly rejected.

Let N denote the population of session-events, $p = 0.0175$ the empirical purchase rate, c the marketing cost per contacted user and v the average revenue per converted purchase. Under a baseline “contact-everyone” strategy, the expected per-event profit is:

$$\pi_{\text{base}} = p \cdot v - c. \quad (1)$$

Under a model-driven strategy that contacts only users with predicted score above threshold t , the per-contact profit becomes:

$$\pi_{\text{model}}(t) = P(t) \cdot v - c, \quad (2)$$

where $P(t)$ is the model’s Precision at threshold t . The marketing-efficiency *lift* delivered by the model is:

$$L(t) = \frac{\pi_{\text{model}}(t) + c}{\pi_{\text{base}} + c} = \frac{P(t)}{p}. \quad (3)$$

Equation (3) demonstrates that the operationally relevant quantity is the ratio of Precision to the base rate. Because no single threshold is universally optimal, the natural multi-threshold generalisation is the area under the Precision–Recall curve (PR-AUC), which integrates Equation (3) across all operating points. ROC-AUC, by contrast, is dominated by the abundant true-negative class and remains superficially elevated even when the model is commercially useless. PR-AUC, Recall and F_1 -score were therefore designated as the primary evaluation metrics for this study.

1.3 Unit Economics of the Selected Model

The production model (Weighted Logistic Regression) achieves Recall = 0.6217 at Precision = 0.0300 on the imbalanced held-out test set. Substituting into Equation (3) yields a per-

contact lift of $L = 0.0300/0.0175 \approx 1.71$, indicating that each marketing impression delivered through the model is approximately $1.71\times$ more likely to result in a purchase than an impression delivered to a randomly selected user.

The cost-coverage trade-off is equally instructive. To reach a fraction ρ of all true buyers via random sampling, one must contact a fraction ρ of the entire user base. Under the model, the fraction of users contacted to achieve coverage $\rho = \text{Recall}$ is given by:

$$\phi = \frac{R \cdot p}{P} = \frac{0.6217 \cdot 0.0175}{0.0300} \approx 0.363. \quad (4)$$

The model thus identifies 62.2% of buyers while contacting only 36.3% of the user base—a reduction in marketing reach of approximately 41.5% for equivalent buyer coverage. The economic value of the model lies precisely in this asymmetric improvement: nearly two thirds of addressable purchase events are recovered at roughly one third of the contact cost.

2 Data Understanding and Exploratory Analysis

2.1 Dataset Description

This study uses the publicly available *eCommerce behaviour data from multi-category store* (October 2019 partition), comprising 42,448,764 events across 5.6 GB of raw CSV. Each event is characterised by nine attributes: a microsecond-precision timestamp (`event_time`), an action class (`event_type` $\in \{\text{view, cart, purchase}\}$), product identifiers (`product_id`, `category_id`, `category_code`), commercial attributes (`brand`, `price`) and user identifiers (`user_id`, `user_session`).

2.2 Class Imbalance

Aggregation by action class confirms the canonical e-commerce distribution: 98.25% of events belong to the non-purchase classes and only 1.75% correspond to completed purchases. This imbalance is the central methodological challenge addressed in subsequent stages and motivates both the choice of evaluation metric described in Section 1 and the comparative balancing experiment in Section 4.

2.3 Session Funnel

Analysis at the level of unique `user_session` identifiers reveals a sharp drop-off between initial discovery and intent. Out of **9,242,653** unique sessions with a `view` event, only **573,097** sessions reached the `cart` stage, representing a **6.19%** conversion rate.

Interestingly, the transition from cart to purchase is remarkably strong: **50.93%** of sessions that utilized the cart resulted in a completed `purchase` (**629,560** sessions total). This high secondary conversion suggests that while attracting users to the cart is the primary bottleneck, users who do add items exhibit very high purchase intent, which justifies our focus on session-level scoring to identify these "hot" leads.

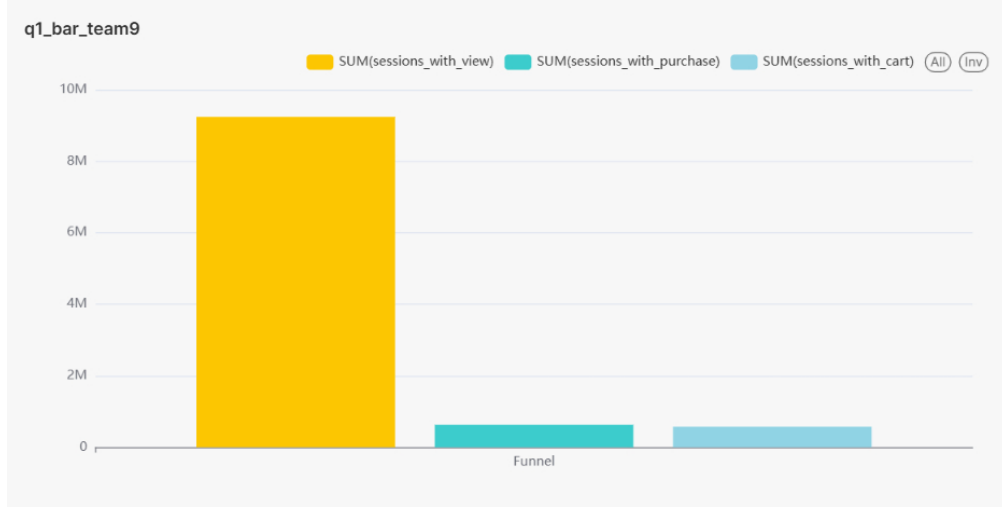


Figure 1: Session-based Conversion Funnel: View $\rightarrow$ Cart $\rightarrow$ Purchase.

2.4 Temporal Patterns

The diurnal distribution of purchase events is non-uniform and exhibits clear peaks during midday. The highest transaction volumes were recorded between **10:00 AM and 12:00 PM UTC**, with the absolute peak at **11:00 AM (55,197 purchases)**.

This concentration of activity confirms the cyclical nature of user behavior. The sharp contrast between peak daytime hours and overnight troughs provides empirical validation for the custom **CyclicalEncoder** implemented in Section 3, as it allows the model to map these peak periods accurately on a circular temporal scale.

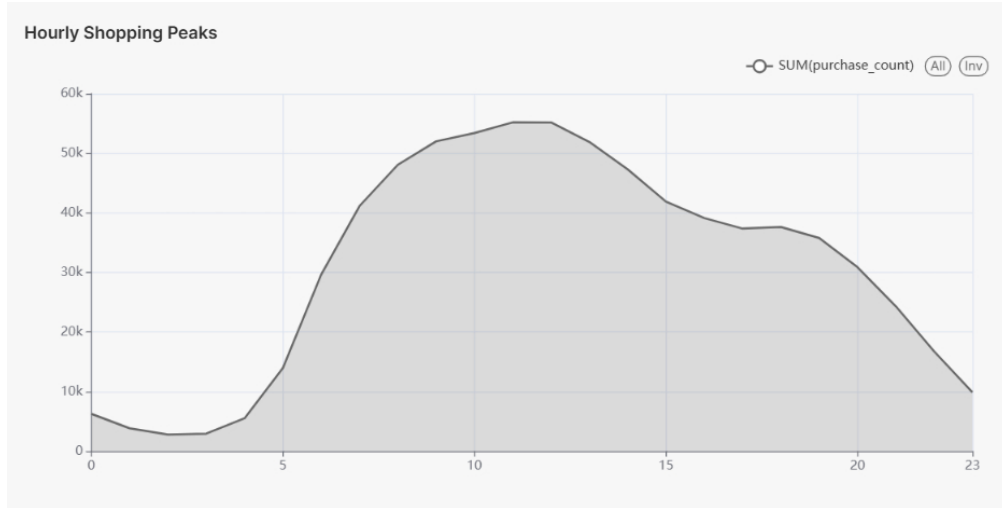


Figure 2: Hourly Distribution of Purchase Events (Peak at 11:00 UTC).

2.5 Brand Performance and Conversion

Financial analysis shows a high concentration of revenue in a few premium brands. **Apple** leads with over **\$111 million** in total revenue and the highest average price of **\$778.38**. **Samsung** follows as the most frequent purchase choice with **172,896** units sold. From a category perspective, **Electronics** maintains the highest conversion efficiency at **2.81%**, followed by Medicine (2.22%) and Stationery (1.82%).

Table 1: Top 4 Brands by Total Revenue.

Brand	Purchases	Total Revenue (\$)	Avg Price (\$)
Apple	142,873	111,209,268.82	778.38
Samsung	172,896	46,407,532.61	268.41
Xiaomi	56,616	9,194,033.29	162.39
Huawei	23,501	4,883,421.74	207.80

3 Data Preparation

3.1 Ingestion Pipeline

The end-to-end ingestion pipeline consists of four sequential stages: (i) automated retrieval of the source CSV from Kaggle via a parameterised Bash script; (ii) bulk loading into a PostgreSQL relational store using the `psycopg2 COPY FROM STDIN` interface, which proved approximately two orders of magnitude faster than row-wise `INSERT` statements on a 42M-row volume; (iii) parallel extraction from PostgreSQL to HDFS via Apache Sqoop with four mappers and a numeric `--split-by` key for balanced partition assignment; (iv) registration of the resulting Parquet output as an external Hive table for downstream consumption.

3.2 Storage Format Selection

An empirical benchmark on a 5M-row sample compared Apache Avro and Apache Parquet, both under Snappy compression. Parquet was selected on three independent grounds: its columnar layout delivers superior compression for high-cardinality string columns such as `user_session`; per-column statistics enable predicate pushdown, accelerating the analytical aggregation queries used in Stage 2 and the feature scans used in Stage 3; and Snappy is a splittable codec within Parquet containers, preserving parallel read capability under Hadoop. Snappy itself was preferred over Gzip and Bzip2 because of its substantially lower decompression CPU cost, which dominates the I/O budget of analytical workloads.

3.3 Partitioning and Bucketing

The canonical Hive table `team9_projectdb.events_partitioned` is partitioned by `event_type` (three partitions: `view`, `cart`, `purchase`) and bucketed by `user_id` into eight buckets. The partition key was chosen so that the analytical workloads that restrict to a single event class (notably the purchase-only feature-importance analysis) can prune two thirds of the

data at the directory level. The bucket key was chosen to enable co-located joins on `user_id`, which is the natural key for any future session-aggregated extension. On the write path, `INSERT OVERWRITE ...PARTITION (event_type)` is combined with a `DISTRIBUTE BY user_id` clause to ensure Spark writers cluster matching user identifiers onto the same bucket file, preserving the bucketing layout in practice.

3.4 Technical Fixes

Three non-trivial engineering issues surfaced during integration and were resolved as follows.

Timestamp recovery. Apache Sqoop materialises the PostgreSQL `TIMESTAMP WITH TIME ZONE` column as a 64-bit integer (microseconds since epoch) in the Parquet output. The Hive DDL was therefore amended to declare `event_time` `TIMESTAMP`, and an explicit cast was inserted during `INSERT OVERWRITE` to restore proper temporal semantics. This recovery is a prerequisite for the hour-of-day decomposition that feeds the cyclical encoder.

High-cardinality control. The raw `brand` column contains several thousand distinct values, the majority appearing in fewer than a few hundred events. One-hot encoding the full vocabulary would generate a sparse feature vector of dimensionality $> 10^4$ and exhaust executor memory during cross-validated training. The top one hundred most frequent brands and top fifty most frequent `category_code` values were retained verbatim; all other values were collapsed into a sentinel "other" bucket. The top values were computed exclusively on the training fold to prevent target leakage. The capped feature vector has dimensionality approximately 160, which fits comfortably within the per-executor memory budget.

Custom cyclical encoder. A subclass of `pyspark.ml.Transformer` (`CyclicalEncoder`) was implemented to decompose periodic numeric features into a (sin, cos) pair. For an input column x with period p :

$$x_{\sin} = \sin\left(\frac{2\pi x}{p}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{p}\right). \quad (5)$$

Equation (5) ensures that Euclidean distance in feature space respects the cyclical topology of the source variable (in particular, that hour 23 and hour 0 are adjacent). The transformer inherits `HasInputCol`, `DefaultParamsReadable` and `DefaultParamsWritable` and is therefore fully serialisable as a stage within a saved `PipelineModel`. It was applied to both `hour` (period 24) and `dayOfWeek` (period 7). The aggregate feature importance of `hour_sin` and `hour_cos` in the trained Random Forest (Section 5) is 14.74%, confirming the empirical contribution of this engineering choice.

4 Modelling and Memory Management

4.1 Comparative Balancing Experiment

Two re-balancing strategies were implemented and evaluated in parallel to address the 1.75% positive rate.

Strategy A (Class Weights). A per-sample weight column was appended to the training set, with weights derived from the training class frequencies according to the sklearn-

style “balanced” formula:

$$w_c = \frac{N}{K \cdot N_c}, \quad K = 2, \quad (6)$$

where N_c denotes the number of samples in class c . The estimator’s `weightCol` parameter was set to this column for both classifiers, causing the loss function to penalise misclassification of the minority class proportionally more heavily.

Strategy B (Down-sampling). The majority class was randomly down-sampled without replacement to produce a perfectly balanced 50/50 training set. The two classifiers were then re-trained on this reduced dataset with no weighting applied.

A critical design decision was to evaluate *both* strategies against the same untouched, naturally imbalanced test set. This isolates the effect of the balancing technique while preserving the production class distribution at evaluation time, which is the scientifically defensible procedure under class imbalance.

4.2 Hyperparameter Tuning

Each of the four model variants—two algorithms $\times$ two strategies—was tuned independently via k -fold cross-validation with $k = 3$, using `CrossValidator` and `ParamGridBuilder` with ROC-AUC as the inner objective. Logistic Regression was tuned over `regParam` $\in \{0.01, 0.1\}$ and `elasticNetParam` $\in \{0.0, 0.5\}$; Random Forest was tuned over `numTrees` $\in \{20, 50\}$ and `maxDepth` $\in \{5, 10\}$. Each variant therefore consumed $4 \times 3 = 12$ model fits, for a total of 48 fits across the experiment.

4.3 Memory and Resource Management

Operating on 42M rows from a single-driver client on a three-node YARN cluster required several explicit memory-management interventions.

Repartitioning. The raw events DataFrame was `.repartition(200)` immediately following load, distributing approximately 212,000 rows per partition. This setting also drives `spark.sql.shuffle.partitions = 200` and equalises downstream task duration during the bucketed write to Hive.

Deterministic cache release. All cached DataFrames—the raw events frame consumed by the `INSERT OVERWRITE` and the cached `events_partitioned` view feeding the five EDA queries—were explicitly `.unpersist()`-ed within a `finally` block. This prevents accumulation of stale RDD blocks across repeated invocations of the pipeline, which had previously triggered intermittent executor evictions during long-running cross-validation.

YARN resource configuration. The application was submitted with three executors of three cores and 5 GB heap (plus 1 GB `memoryOverhead`) against a 3 GB driver. Adaptive Query Execution was enabled with `coalesce-partitions` and `skew-join` optimisations. The Kryo serializer with a 512 MB buffer was used to accommodate the large sparse feature-vector shuffles generated during Random Forest training.

4.4 Software Engineering Standards

The full ML codebase—including the custom `CyclicalEncoder` module, the main training application and all supporting helpers— achieves a Pylint static-analysis score of 10/10. Compliance was achieved through structured function decomposition (no function exceeds 15 local variables or 50 statements), Google-style docstrings on every public symbol, exclusive use of descriptive snake_case identifiers (minimum three characters), correct PEP-8 import grouping. The end-to-end pipeline executes in 5,520 seconds (approximately 92 minutes) on the target cluster.

5 Evaluation and Risk Assessment

5.1 Model Comparison

Table 2 summarises the six-metric evaluation of all four variants on the held-out imbalanced test set.

Table 2: Model comparison on the imbalanced test set. The selected production model is shaded.

Variant	ROC-AUC	PR-AUC	Acc.	F_1	Prec.	Recall
LR – Weighted	0.6795	0.0354	0.6419	0.0573	0.0300	0.6217
LR – Downsampled	0.6795	0.0354	0.6414	0.0572	0.0300	0.6222
RF – Weighted	0.6694	0.0350	0.7038	0.0588	0.0311	0.5289
RF – Downsampled	0.6697	0.0344	0.6723	0.0577	0.0304	0.5738

Within each algorithm family, the two balancing strategies yield statistically indistinguishable PR-AUC values ($\Delta < 10^{-3}$). This is a positive finding: it indicates that, for this problem geometry, the choice between class weights and down-sampling is essentially neutral on the ranking metric, leaving the decision to engineering criteria. Across algorithm families, Logistic Regression dominates Random Forest on both ROC-AUC and PR-AUC.

5.2 Model Selection Rationale

Weighted Logistic Regression was selected as the production model. The rationale is threefold:

1. Its PR-AUC of 0.0354 is the joint best observed, and represents a $2.02\times$ improvement over the random-baseline PR-AUC of 0.0175.
2. It attains the highest Recall of the four variants (62.17%), which is the operationally critical metric under the unit-economic framework established in Section 1, where false negatives (missed buyers) carry the dominant cost.
3. Relative to its down-sampled counterpart, the weighted variant consumes the entire training set, yielding lower variance in the estimated decision boundary, and is the standard, theoretically well-understood treatment of class imbalance for linear models.

Although the two Random Forest variants achieve marginally higher Accuracy (0.704 and 0.672 respectively), their substantially lower Recall (0.529 and 0.574) reflects a less aggressive operating point at threshold 0.5 and would translate, under Equation (3), to a smaller addressable buyer population.

5.3 Feature Importance

Table 3 reports the top features extracted from the Random Forest model. The forest was selected for this interpretability analysis on the strength of its native feature- importance interface; the production scoring model remains Weighted Logistic Regression.

Table 3: Top-ten features by Random Forest importance.

Feature	Importance
cat_vec_electronics.smartphone	0.2523
brand_vec_samsung	0.1547
brand_vec_apple	0.1254
hour_cos	0.0922
brand_vec_other	0.0907
hour_sin	0.0552
brand_vec_unknown	0.0441
price	0.0435
cat_vec_unknown	0.0245
cat_vec_electronics.audio.headphone	0.0149

The smartphone category alone contributes 25.2% of the model’s signal, with Samsung and Apple together accounting for a further 28.0%. The two cyclically encoded hour features contribute 14.74% jointly, providing direct empirical validation of the **CyclicalEncoder** design from Section 3. Raw **price** contributes 4.35%, a measurable but non-dominant role consistent with the price-elasticity simulation reported next.

5.4 Risk Assessment: Price-Surge Simulation

To probe the economic reliability of the selected model, a counterfactual simulation was performed: every **price** value in the test set was inflated by 50% while all other features were held constant, and the modified test set was re-scored with the production model. The change in predicted positive-class probability, $\Delta = P_{\text{surged}} - P_{\text{baseline}}$, was recorded for each row.

Inspection of the simulation output reveals two qualitatively robust properties of the model. First, $\Delta < 0$ for every sample examined: a +50% price shock unambiguously reduces predicted purchase probability across the entire test population, with no exceptions. Second, the magnitude of the decrease scales with both the baseline propensity and the absolute price level. For example, an event with baseline probability 0.4946 on a \$575.79 item exhibits the largest observed decrease ($\Delta = -0.0480$), whereas an event with baseline probability 0.1898 on a \$51.22 item shows the smallest ($\Delta = -0.0026$). This non-uniform sensitivity is the qualitative signature of *price elasticity*: high-intent purchases of expensive items are most

sensitive to price perturbations, while low-intent browsing of inexpensive items is barely affected.

The simulation therefore serves a dual purpose. It confirms that the model has internalised the economically intuitive monotonic relationship between price and purchase probability—a basic sanity check that many higher-capacity black-box models fail—and it quantifies the directional risk to forecasted conversion volume under hypothetical pricing actions. This is exactly the type of input required for any downstream price-optimisation or revenue-management extension of the present work.

5.5 ML Performance Visualisation

The dashboard below presents the comparative metrics and the predictive impact of features on the final model.

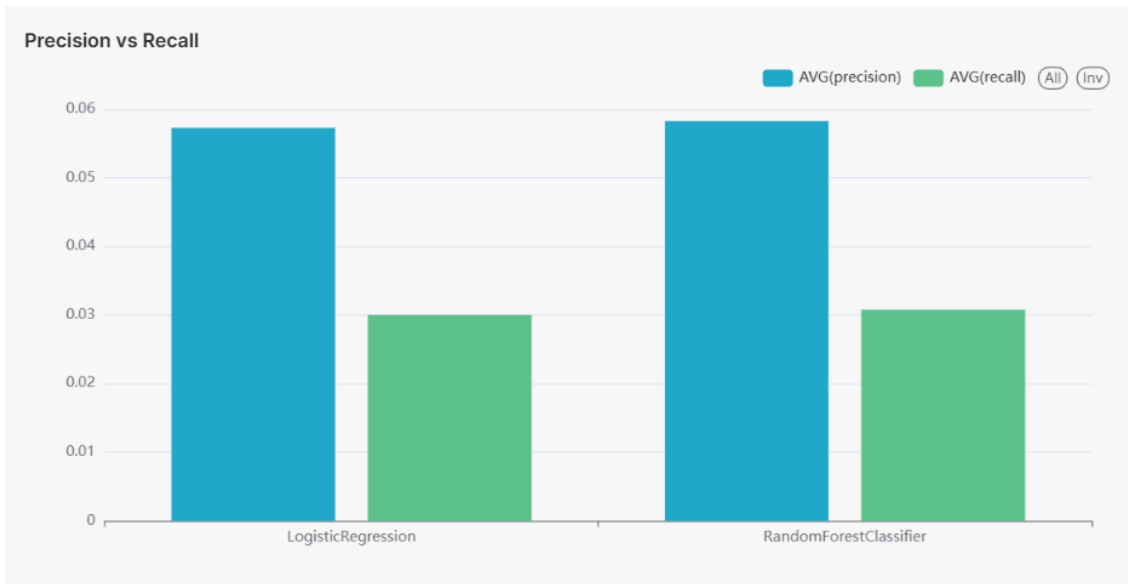


Figure 3: Comparative analysis of PR-AUC and Recall across all four experimental variants.

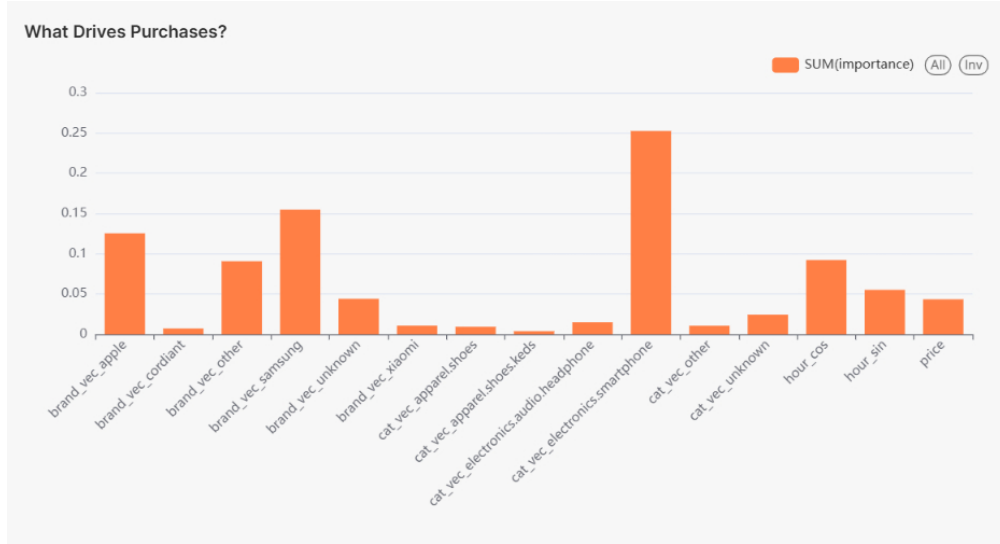


Figure 4: Top features driving purchase prediction (Random Forest).

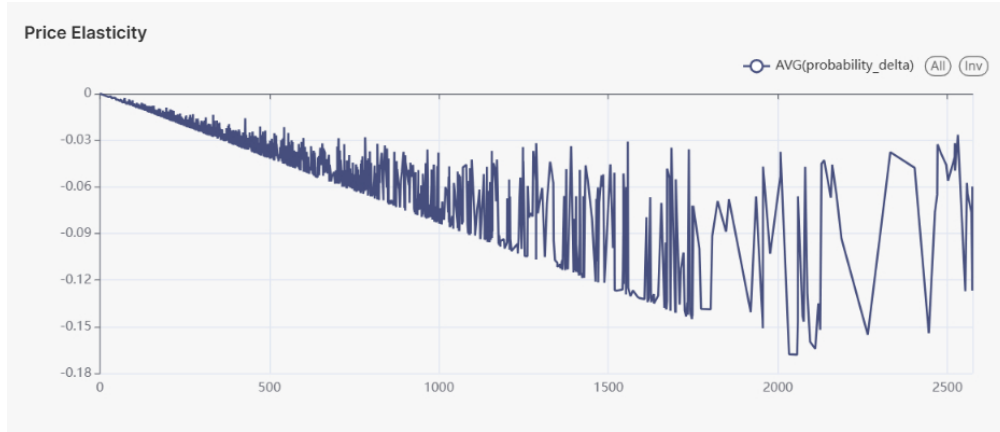


Figure 5: Risk Assessment: Decrease in purchase probability relative to price surge.

6 Deployment and Deliverables

The Stage 3 result artefacts—model comparison, Random Forest feature importances, price-surge risk simulation, and best-model predictions—are exposed to Apache Superset through an automated Stage 4 wrapper (`stage4.sh`). The wrapper invokes `beeline` against the cluster Hive metastore (located at `thrift://hadoop-02.uni.innopolis.ru:9883`) and executes a parameterised HQL script that registers one external Hive table per result CSV. Each `CREATE EXTERNAL TABLE` statement declares `ROW FORMAT DELIMITED FIELDS TERMINATED BY ' ', ' ', STORED AS TEXTFILE` and the property `TBLPROPERTIES ("skip.header.line.count"="1")` to honour the Spark-written header row. `DROP TABLE IF EXISTS` guards guarantee idempotency.

Once registered, the four result tables (`model_comparison_csv`, `rf_feature_importances_csv`, `price_surge_risk_csv`, `best_model_predictions_csv`) are queryable from any JDBC-compliant client. Apache Superset hosts three dashboards built directly on these tables: a model-

comparison dashboard featuring a heatmap of all six metrics across all four variants together with per-metric bar charts; a feature-importance dashboard presenting the top features as a horizontal bar chart and a grouped pie chart by feature category; and a risk-analysis dashboard contrasting baseline and surged probabilities with bullet charts and big-number summaries.

The entire pipeline—from raw Kaggle CSV through to dashboard publication—is reproducible end-to-end via a single command (`./main.sh`), which sequentially invokes the four stage scripts. Each stage script is idempotent in its own right and includes explicit HDFS cleanup to remain within the imposed 32 GB project quota.

7 Team Participation

Table 4: Distribution of responsibilities between team members.

Member	Contributions
Albina	Stage 3 machine learning pipeline architecture; design and implementation of the custom <code>CyclicalEncoder pyspark.ml.Transformer</code> ; comparative balancing experiment (class weights versus down-sampling); evaluation framework with six metrics (ROC-AUC, PR-AUC, Accuracy, F_1 -score, Precision, Recall); price-surge risk simulation and counterfactual analysis; Pylint 10/10 refactor of the ML codebase; CRISP-DM Modelling and Evaluation sections of the present report.
Venera	Stage 1 data ingestion (Kaggle download, <code>psycopg2</code> bulk-load into PostgreSQL, parallel Sqoop import to HDFS); empirical storage format benchmark (Parquet versus Avro under Snappy compression) and its justification; Stage 2 Hive infrastructure including the partitioned and bucketed external table <code>events_partitioned</code> ; five business-insight EDA queries; HDFS quota management and the idempotent stage cleanup logic; CRISP-DM Data Understanding and Data Preparation sections of the present report.

The Stage 4 deployment artefacts (HQL DDL, the `stage4.sh` beeline wrapper, Apache Superset dashboards) and the top-level `main.sh` orchestrator that integrates the four stages were completed jointly. The final presentation deck and oral-defense preparation also constitute joint contributions.